Team Name: TREE(3)

Team Members: Muhammad Abdullah Abid, Peter Batarseh, Nadeem Taysun

Project Title: An implementation and comparison of two file tree systems

Problem: How do we store and access files in a file tree in an efficient manner?

Motivation: Files systems are a necessary component of all computer usage, and knowing which data structure accesses/stores files in the quickest way possible.

Features: The problem will have been solved when the implementation(s) are able to search/access elements. The optimal candidate will be determined via which one has the least node accesses involved. Time complexity will also be found and accounted for.

Data: A generated file tree that contains folders which contains folders or files. The file tree is represented as a C++ struct, with folders/files also being structs. Each file/folder is represented with a unique name. This is to prevent polluting the actual desktop with dummy files.

Tools: C++ with Catch framework for unit tests. HTML/CSS for rendering the frontend
.
Visuals:

1. Selection between B+ tree and splay tree implementation

2. A frame that contains: the current directory you are in, the # of sub-directiories, and the # of files in the current directory. A counter for the total number of node accesses.

Strategy: We will create two implementations of a file tree. The first implementation is a B+ tree, where nodes all contain directory information. The leaves contain the data/files. The second implementation is a n-ary tree that contains a splay tree for all files of a given directory; a tree of splay trees.

Distribution of Responsibility:

Peter Batarseh: File/folder struct/class, B+ tree/implementation 1, unit tests (implementation 1)

Nadeem Taysun: N-ary and splay tree component of implementation 2. Unit test (implementation 2).

Muhammad Abdullah Abid: Web Frontend, Connecting the frontend to the backend, and Data set generation.



